How the profile files are arranged for the model, in plain language

This page explains what `optimize_profile_files.py` (step 13 of the chain) does and why, for someone who has not worked with the MITgcm profiles package before. No background is assumed beyond "the model reads our profile files". If a step seems odd, there is usually a good reason, and the reason is written down here.

---

1. The problem in one paragraph

The ocean model is split into many small square patches of ocean, called **tiles**, and each tile is handled by its own processor. When the model reads our profile files, it sends every profile to the tile it falls in. Before it can do that, each tile has to **reserve memory** for the profiles it might receive, and it has to reserve the *same amount* as every other tile, sized for the worst case. Most of that reserved memory ends up unused. Our job is to arrange the profiles into files so that the reserved amount is as small as it can sensibly be.

---

2. Tiles: where the "113" (or "117", or "96") comes from

The model grid (LLC90) is made of **13 square faces**, each 90 × 90 grid cells. For computing, each face is cut into tiles of **30 × 30 cells**, so each face becomes 3 × 3 = 9 tiles:

> 13 faces × 9 tiles = **117 tiles**

Tiles that are entirely land are usually switched off ("blank") in a model run. ECCO v4r6 switches off 4 of them, leaving 113; the older v4r4 switched off 21, leaving 96. **This program does not need to know which tiles are switched off.** A profile is in whatever tile it is in; if the model has switched that tile off, the model simply ignores the profile. All the program needs is the grid size (90) and the tile size (30).

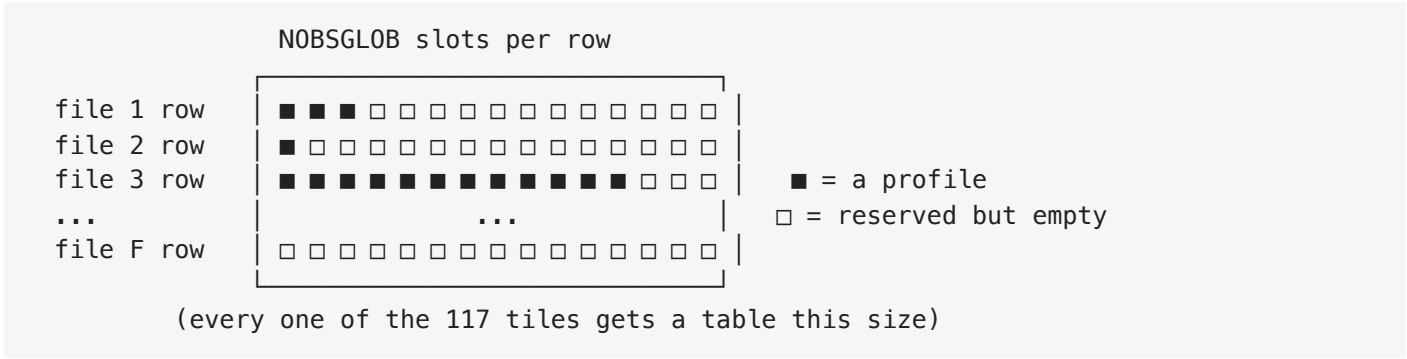
---

3. What the model reserves: two numbers

For every tile, the model reserves a block of memory shaped like a table:

- **one row per profile file** — the number of rows is `NFILESPROFMAX` (the number of files), and
- **a fixed number of slots per row** — `NOBSGLOB`, which must be at least the largest number of profiles *any one file* puts into *any one tile*,

and each slot holds a whole profile (all its depth levels).



Two things are worth noticing:

1. **`NOBSGLOB` is not a limit on how big a file can be.** A file can hold millions of profiles. It only matters how many of them land in the *same* tile.
2. **Every tile gets the same size of table**, even a tile in the middle of the empty Southern Ocean, because the table is sized by the single busiest (file, tile) combination anywhere.

So the memory each tile reserves is proportional to

> **`NFILESPROFMAX` × `NOBSGLOB`** (× the number of depth levels)

and **making that product small is the whole goal**. Because the number of profiles is fixed, a smaller product simply means **less empty space**.

---

4. The rules we must keep

The files are not free-form; a few rules come from how the data are used:

Rule	Why
One provenance per file (all Argo in Argo files, all XBTs in XBT files, …; ITP L2 and L3 kept apart)	The model's weighting knobs in `data.profiles` are set per file, so mixing sources in one file would mix their weights.
Chronological, one continuous time span per file	Easier to manage, and it is how ECCO's own files are organised.
Files labelled in whole calendar years (start 1 Jan, end 31 Dec)	Easier internal bookkeeping. A file can hold several whole years (e.g. `PFL_001_19970101_20131231`). If one single year is too dense, that year is split into parts that each carry the year's label (`XBT_WOD_001_19920101_19921231`, `XBT_WOD_002_19920101_19921231`). A file never mixes part of a year with other years.

---

5. The tug-of-war

There are two ways to make the reserved table smaller, and they pull against each other:

- **Fewer files** (fewer rows) → but then each file has to hold more profiles, so busy tiles get more profiles per file, so `N0BSGL0B` (row length) goes up.
- **Smaller `N0BSGL0B`** (shorter rows) → but then a file has to stop as soon as its busiest tile fills up, so you need more files.

Our data have a few extremely busy tiles. The busiest is in the western North Pacific (South China Sea / Philippine Sea / around Japan), where CTD casts alone put 168,201 profiles in one tile. A tile like that forces either a long row or many files.

There is also a **hard floor**: whatever you do, `NFILESPR0FMAX` × `N0BSGL0B` cannot go below the sum of each provenance's busiest-tile count — about **600,000** for the 2026-10-12 data. (Each provenance's busiest tile has to fit, spread over that provenance's files.)

---

6. How the program finds a good answer

The program does something simple and dependable:

1. **For every candidate `N0BSGL0B`** (it tries them all, in small steps), it asks: *what is the smallest number of files that keeps every tile in every file at or below this value, while keeping the rules above?*
2. It answers that by walking through each provenance's profiles in time order and starting a new file only when it has to (when adding the next year, or the next profile of a too-dense year, would push some tile over the limit). Starting a new file only when forced is what gives the fewest files.
3. **It multiplies** files × `N0BSGL0B` for each candidate, giving the size of the reserved table.
4. **It picks the winner.** By default ("auto") it does not insist on the absolute smallest product, because near the bottom the curve is very flat and the true minimum usually needs many more files for a gain of a few percent. Instead it takes **the fewest files whose product is within 5% of the smallest one found**. You can change the 5% (`--tolerance`), cap the number of files (`--max_files`), or just fix `N0BSGL0B` yourself.
5. **It tightens the number.** The candidates are tried in steps (of 250), so the chosen limit is a round number such as 43,500. Once the files are laid out, the busiest (file, tile) actually holds a few less (43,469 here), and that exact value is what it tells you to put in `PR0FILES_SIZE.h`.

For the 2026-10-12 data the curve looks like this (whole-year rule):

NOBSGLOB	Files	Files × NOBSGLOB	Reserved space that is empty
168,201	8 (one per provenance)	1,345,608	96.2%
97,000	10	970,000	94.8%
52,250	16	836,000	93.9%
43,469	**18**	**782,442**	**93.5%** ← chosen ("auto")
36,750	21	771,750	93.4%
28,250	27	762,750	93.3% (the smallest found)

(Each run prints this table for its own data, from the fewest files up, with the chosen layout and the smallest product marked.)

---

7. "93% empty?! Is something wrong?"

No. It sounds alarming, but it is expected, and no arrangement of files can fix it. The data are very unevenly spread over the globe — one tile has 330,655 profiles, the median tile has a few percent of that, and many tiles have almost none — yet every tile must reserve the same size of table. Even the mathematically best possible arrangement would still be about 91% empty.

What the optimisation *does* buy is a much smaller table than a naive arrangement: about **782,000 slots per tile instead of 1,350,000** (one file per provenance), i.e. roughly 40% less reserved memory.

---

8. Reading the output

When it writes files (`--write <dir>`, or automatically in step 13 into `<run>/ecco_arrays/`), that folder holds only the profile files the model reads. The summary files go in a `documentation/` folder beside it (`<run>/documentation/`):

- **`PROFILES_SIZE_and_data.profiles.txt`** — the two numbers to put in the model's `PROFILES_SIZE.h` (`NOBSGLOB`, `NFILESPROFMAX`) and the `profilesfiles(...)` lines for `data.profiles`. You can copy these directly.
- **`ecco_arrays_manifest.csv`** — one line per file: provenance, years covered, number of profiles, and its busiest-tile count.
- **`optimize_sweep.csv`** — the whole trade-off curve, if you want to plot it or pick a different point.

Each output file also remembers where every profile came from (`prof_yearly_file`, `prof_yearly_index`), so nothing is lost or untraceable.

To check the result: `python diagnostics/check_ecco_arrays.py <run_dir>` (every profile packed exactly once, data identical, order and labels correct, no tile over the limit).

---

9. Common questions

Why not just one file per provenance? That gives the fewest files, but the busiest tile then needs very long rows (168,201 slots), and every tile pays for that: ~1.35 million slots each, the worst option in the table.

Why not a tiny `NOBSGLOB`, like 1,000 or 100? You would need hundreds or thousands of files, and the product does not shrink — it actually grows (files close while most of their rows are still nearly empty). At 1,000 you need 747 files; at 100, 8,460.

What happens when we add new data? Run it again; it recomputes everything. The numbers to put in `PROFILES_SIZE.h` may change, and the program prints them.

Can I use a different grid, e.g. LLC270? Yes: `python optimize_profile_files.py <dir> --llc 270 --tile 30 --grid_dir <llc270 grid>`. (For a grid the chain did not run on, add `--position obs` so profiles are placed by their own latitude/longitude; this is the default for any grid other than LLC90.)

I want exactly N files / exactly this `NOBSGLOB`. Use `--max_files N`, or `--nobsglob <value>`.

Do I need to know which tiles are blank, or the bathymetry? No — see section 2.